Application de Gestion des Stages

Etude de cas : DevTec

Document explicatif : architecture, fonctionnalités, règles métier, tests et installation

Spring Boot 3 • Angular 19 • MySQL • JUnit 5

Faculté des Sciences Economiques et de Gestion de Sfax
Licence Fondamentale en Informatique (LFI 3) - Génie Logiciel
Année universitaire 2026 / 2027

1. Présentation du projet

DevTec accueille chaque année (janvier et juin) des stagiaires. Le processus actuel est manuel : les informations sont dispersées dans des documents manuscrits, ce qui rend difficile le suivi des stagiaires. L'application centralise tout le processus : la proposition des projets par les équipes, la réception des candidatures, leur tri automatique, l'envoi des emails d'acceptation et la génération des conventions de stage.

Acteurs du système

Acteur	Rôle dans l'application
Candidat (visiteur)	Consulte le catalogue des stages et dépose sa candidature, sans compte.
Agent RH	Gère les projets, mentors, compétences et niveaux ; examine les candidatures triées ; accepte ou refuse ; génère les conventions.
Mentor	Consulte les projets qu'il encadre, ses stagiaires et leurs conventions.
Administrateur	Crée et supprime les comptes et attribue les rôles. Il a aussi accès aux fonctions RH.

Technologies utilisées

Partie	Technologies
Backend	Java 17, Spring Boot 3.2, Spring Web (API REST), Spring Data JPA / Hibernate, Spring Security (HTTP Basic + BCrypt), Bean Validation, Spring Mail, Lombok
Base de données	MySQL 8 (tables générées automatiquement à partir des entités JPA)
Frontend	Angular 19 (composants standalone), Bootstrap 5, Bootstrap Icons
Tests	JUnit 5 + Mockito

2. Architecture

L'application suit une architecture client / serveur. Le frontend Angular appelle l'API REST du backend Spring Boot, qui accède à la base MySQL. Le backend est organisé en couches pour séparer les responsabilités :

Package	Contenu
entity	Les classes du diagramme de classes, annotées JPA (@Entity, @OneToMany, @ManyToMany...)
dto	Les objets échangés avec le frontend (on n'expose jamais directement les entités)
mapper	Conversion entité vers DTO et DTO vers entité
repository	Interfaces Spring Data JPA (accès à la base)
service	La logique métier : score, tri, numéro de convention, email...
controller	Les points d'accès REST (/api/...)
config	Configuration de la sécurité et données initiales (comptes de test)
exception	Exceptions métier + gestionnaire global qui renvoie des messages d'erreur clairs

Pourquoi des DTO et des mappers ?

- Les entités contiennent des relations bidirectionnelles (ex. Candidature / ConventionStage) qui provoqueraient des boucles infinies en JSON.
- Le frontend reçoit uniquement les données utiles (ex. mentorNom au lieu de l'objet Mentor complet).

- Les données sensibles ne sortent jamais (le mot de passe du compte n'est jamais renvoyé).
- La validation (@NotBlank, @Email, @NotEmpty...) se fait sur les DTO reçus.

Exemple de flux : accepter une candidature

```
Angular → PUT /api/candidatures/5/accepter → CandidatureController →
CandidatureService.changerStatut() → CandidatureRepository.save() →
EmailService.envoyerAcceptation() → CandidatureMapper.toDto() → réponse JSON
```

3. Modèle de données (diagramme de classes)

Les entités reprennent le diagramme de classes fourni. Pour couvrir tout l'énoncé, trois compléments ont été ajoutés : la classe abstraite Personne (informations personnelles), l'attribut dateLimiteCandidature du projet et l'etat des tâches.

Classe	Attributs	Relations
Compte	login, mdp, role	Associé à un AgentRH ou à un Mentor (XOR)
Personne (abstraite)	nom, prenom, adresse, email, telephone	Classe mère de AgentRH et Mentor
AgentRH	(hérités de Personne)	0..1 Compte
Mentor	specialite (+ hérités)	0..1 Compte ; encadre 0..* ProjetStage
ProjetStage	titre, dateDebut, dateFin, dateLimiteCandidature, objectifs	1 Mentor ; composition 0..* Tâche ; 1..* NiveauEtudes ; 0..* Compétence (via CompetenceProjet)
Tâche	libelle, etat	Appartient à 1 ProjetStage (supprimée avec le projet)
Compétence	libelle	Liée aux projets et aux candidatures
CompetenceProjet	poids	Classe d'association ProjetStage - Compétence
NiveauEtudes	niveau	Licence, Mastère, Ingénieur
Candidature	nomCandidat, prenomCandidat, email, telephone, adresse, dateCandidature, statut, /score	1 ProjetStage ; 1 NiveauEtudes ; 1..* Compétence ; 0..1 ConventionStage
ConventionStage	numero, dateCreation	1 Candidature
StatutCandidature	EN_ATTENTE, ACCEPTEE, REFUSEE	Enumération
EtatTache	A_FAIRE, EN_COURS, TERMINEE	Enumération (suivi des progrès)

/score est un attribut dérivé : il n'est pas stocké en base, il est calculé à chaque lecture par le service. La base contient 12 tables : agentrh, compte, mentor, competence, niveau_etudes, projet_stage, tache, competence_projet, projet_niveau, candidature, candidature_competence, convention_stage.

4. Règles métier

Calcul du score et tri automatique

```
score = somme des poids des compétences du projet que possède le candidat
+ 10 si le niveau d'études du candidat fait partie des niveaux demandés
```

Exemple : un projet demande Android (poids 5) et UI/UX (poids 3) pour les niveaux Licence et Ingénieur. Sirine (Licence, Android + UI/UX) obtient 5 + 3 + 10 = 18. Ali (Mastère, UI/UX) obtient 3. Sirine apparaît donc en premier dans la liste du RH.

Numéro unique de la convention

Format imposé par l'énoncé : numéro séquentiel - mois de recrutement (1 ou 6) - année. Le mois vaut 1 si le stage commence avant juin, 6 sinon. Le numéro séquentiel est calculé en comptant les conventions existantes de la même session. Exemple : 03-6-2026 = troisième stagiaire recruté en juin 2026.

Délai de réception des candidatures

Pour chaque projet, le RH fixe une date limite de candidature, obligatoirement avant le début du stage. Le catalogue affiche « Candidatures jusqu'au ... » et le bouton Postuler disparaît après cette date. Le backend refuse aussi toute candidature reçue après la date limite.

Suivi des progrès des stagiaires

Chaque tâche du projet a un état : A faire, En cours ou Terminée. Le mentor met à jour l'état depuis « Mes projets » (il ne peut modifier que les tâches de ses propres projets). L'avancement du projet = tâches terminées / nombre de tâches x 100, affiché avec une barre de progression pour le mentor et pour le RH. Quand le RH modifie le projet, l'état des tâches existantes est conservé.

Informations personnelles

AgentRH et Mentor héritent de la classe abstraite Personne (nom, prénom, adresse, email, téléphone). Ces informations sont saisies à la création du compte, le RH gère celles des mentors, et chaque utilisateur peut modifier les siennes dans « Mon profil ». Le candidat fournit aussi son prénom, son téléphone et son adresse, qui apparaissent dans la convention (comme dans l'annexe 2).

Autres contrôles

- Le numéro de téléphone du candidat doit contenir 8 à 15 chiffres.
- Une convention n'est générée que pour une candidature acceptée, et une seule fois.
- On ne peut pas changer le statut d'une candidature qui a déjà une convention.
- On ne peut pas supprimer un projet qui a des candidatures, ni un mentor qui encadre des projets.
- La date de fin d'un projet doit être après la date de début ; un login de compte doit être unique.

Email d'acceptation

Quand le RH accepte une candidature, un email est envoyé au candidat (titre du projet, dates, mentor). Si l'envoi échoue (ex. SMTP non configuré), la candidature reste acceptée et l'interface affiche un avertissement. L'envoi utilise Gmail : il faut renseigner l'adresse et un « mot de passe d'application » dans application.properties.

5. Fonctionnalités par écran

Ecran	Accès	Description
Catalogue	Public	Liste des projets avec dates, mentor, niveaux, compétences ; badge Ouvert / Fermé
Postuler	Public	Formulaire : nom, prénom, email, téléphone, adresse, niveau d'études, compétences
Connexion	Public	Login / mot de passe, redirection selon le rôle
Projets	RH, Admin	Liste avec date limite et barre d'avancement ; ajout, modification, suppression ; fiche avec date limite, tâches, compétences (poids), niveaux et mentor

Ecran	Accès	Description
Candidatures	RH, Admin	Choix du projet, liste triée par score, accepter (email) / refuser, générer la convention
Conventions	RH, Admin	Liste des conventions et affichage imprimable (modèle de l'annexe 2)
Mentors	RH, Admin	Gestion des mentors et de leurs informations personnelles
Compétences / Niveaux	RH, Admin	Gestion (ajout, modification, suppression)
Comptes	Admin	Création d'un compte (rôle + informations personnelles) ; suppression
Mes projets	Mentor	Suivi des tâches (A faire / En cours / Terminée), avancement, stagiaires, conventions
Mon profil	Connecté	Consultation et modification de ses informations personnelles

Sécurité

- Mots de passe chiffrés avec BCrypt dans la table compte.
- Chaque requête Angular envoie l'en-tête Authorization (HTTP Basic) grâce à un intercepteur.
- Les droits sont vérifiés côté serveur (SecurityConfig) et côté client (guard Angular par rôle).
- CORS autorisé uniquement pour http://localhost:4200.

Principaux points d'accès de l'API

Méthode et URL	Rôle
GET /api/projets - GET /api/projets/{id}	Public
POST / PUT / DELETE /api/projets	RH, Admin
POST /api/candidatures	Public
GET /api/candidatures/projet/{id} (triées par score)	RH, Admin
PUT /api/candidatures/{id}/accepter - /refuser	RH, Admin
POST /api/conventions/candidature/{id}	RH, Admin
GET /api/projets/mes-projets - /api/conventions/mes-conventions	Mentor
GET / POST / DELETE /api/comptes	Admin
PUT /api/taches/{id}/etat?etat=TERMINEE	Mentor (ses projets), RH, Admin
GET /api/auth/login - GET / PUT /api/auth/profil	Utilisateur connecté

6. Tests unitaires (JUnit 5 + Mockito)

Les services sont testés de manière isolée : les repositories et le service d'email sont simulés avec Mockito (@Mock), les mappers sont réels (@Spy). Résultat : 21 tests, 0 échec.

Classe de test	Méthode testée	Ce qui est vérifié
CandidatureServiceTest	calculerScore	Toutes les compétences + bon niveau ; niveau non demandé ; aucune compétence correspondante
	findByProjetTrieesParScore	Les candidatures sont renvoyées du meilleur au plus faible score
	postuler	Statut EN_ATTENTE, date du jour, score, informations personnelles ; refus si délai dépassé

Classe de test	Méthode testée	Ce qui est vérifié
	estOuvert	Ouvert jusqu'à la date limite incluse, fermé le lendemain
	changerStatut	Acceptation : email envoyé ; refus : aucun email
ConventionStageServiceTest	genererNumero	01-6-2026, 03-6-2026, 10-1-2027
	generer	Convention créée pour une candidature acceptée ; refus si non acceptée ou si déjà existante
ProjetStageServiceTest	toDto (avancement)	1 tâche terminée sur 2 = 50 %
	changerEtatTache	Par son mentor : OK ; par un autre mentor : refusé ; par le RH : OK
	remplirProjet	Date limite après le début : refusé ; l'état des tâches existantes est conservé

```
Lancer les tests : cd backend puis mvn test
```

7. Installation et lancement

Prérequis : Java 17, Maven, Node.js 20+, Angular CLI, MySQL.

1) Base de données

La base gestion_stages est créée automatiquement. Identifiants configurés dans backend/src/main/resources/application.properties :

```
spring.datasource.username=stage
spring.datasource.password=stage123
```

2) Backend (port 8080)

```
cd backend
mvn spring-boot:run
```

3) Frontend (port 4200)

```
cd frontend
npm install
ng serve
```

Ouvrir ensuite http://localhost:4200

Comptes de test (créés au premier démarrage)

Login	Mot de passe	Rôle
admin	admin123	Administrateur
rh	rh123	Agent RH
ahmed	ahmed123	Mentor (Ahmed - Développement mobile)

Scénario de démonstration

- Se connecter en rh et créer un projet (mentor Ahmed, compétences avec poids, niveaux, tâches).
- Se déconnecter, aller sur le catalogue et déposer 2 ou 3 candidatures avec des profils différents.
- Revenir en rh : la page Candidatures affiche la liste triée par score.
- Accepter le meilleur candidat (email envoyé), puis générer sa convention (ex. 01-6-2026) et l'imprimer.
- Se connecter en ahmed : passer des tâches à « Terminée » et voir l'avancement monter.
- Toujours en ahmed : compléter ses informations dans « Mon profil ».
- Se connecter en admin : créer un nouveau compte mentor ou agent RH.

8. Couverture de l'énoncé

Besoin de l'énoncé	Etat	Remarque
Gestion des projets de stage (titre, durée, objectifs, tâches, niveaux, compétences, mentor)	Fait	
Soumission des candidatures	Fait	Catalogue public, sans compte
Tri automatique selon le niveau et les requis du projet	Fait	Score avec poids des compétences
Emails d'acceptation	Fait	Nécessite la configuration du compte Gmail
Convention avec identifiant unique numéro-mois-année	Fait	Imprimable, modèle annexe 2
Un mentor attribué à chaque projet / stagiaire	Fait	
Comptes utilisateurs et rôles (administrateur)	Fait	Création et suppression
Sécurité et restriction d'accès selon le rôle	Fait	Spring Security + guards Angular
Modèle relationnel (BD) et méthodes Java du module « gérer stages »	Fait	MySQL, 12 tables
Tests unitaires JUnit	Fait	21 tests
Délai de réception des candidatures	Fait	Date limite fixée par le RH pour chaque projet
Informations personnelles (nom, prénom, adresse, email, téléphone)	Fait	Classe Personne ; gestion par le RH et page « Mon profil »
Suivi des progrès des stagiaires	Fait	Etat des tâches et pourcentage d'avancement
Publication sur les réseaux sociaux / signatures	Hors application	Remplacé par le catalogue public ; signatures manuscrites sur la convention imprimée